Cache Hierarchy Performance Analysis: Normalized Cache Gain and Pareto-Optimal Design

Cache Size, Set Associativity, and Novel Design Metrics

Authors

Arrnav Trisal (24BVD0064)

Gavin Fernandez (24BVD0083)

Ashutosh (24BVD0065)

Varun V Nair (24BVD0061)

Supervisor

Vikas Vijayvargiya

Cache Hierarchy Performance Analysis: Normalized Cache Gain and Pareto-Optimal Design

Cache Size, Set Associativity, and Novel Design Metrics

ABSTRACT

This study looks at how cache memory design affects processor performance, with a focus on finding the best balance between cache size and hardware cost. We use the gem5 simulator to run two sets of experiments: one that changes the size of the L1, L2, and L3 caches, and another that changes in how many ways each cache is set-associative. We measure six performance metrics — 1.Instructions Per Cycle (IPC), 2.Cycles Per 3.Instruction (CPI), 4.miss rate, 5.miss latency, and 6.execution time — to get a full picture of how caches behave under a streaming memory workload.

We introduce two new analytical tools: the Normalized Cache Gain (NCG) metric, which tells us how much IPC improvement we get per kilobyte of cache added; and a Pareto-frontier analysis, which finds the most cost-efficient cache design across the full range of configurations we tested. Our results prove that the L2 cache gives the best return on investment with a peak NCG of 1.1512×10^{-5} IPC/KB, that there is a clear threshold at 8-way L1 associativity where conflict misses disappear, and that the best overall design is L1=32 KB, L2=512 KB, L3=8 MiB which reaches 98.2% of peak performance at only 26.4% of the maximum cache area tested.

Keywords — *gem5, cache hierarchy, IPC, miss rate, set associativity, Normalized Cache Gain, Pareto frontier, memory hierarchy design.*

1. INTRODUCTION

IN the current era, processors are very fast, but main memory is much slower. This gap is often reffered to as the "memory wall" — means that a processor can end up waiting for data most of the time if the cache is not designed well. Caches are small, fast memory banks placed between the processor and main memory to reduce this waiting time. Getting the cache design right is one of the most important decisions in computer architecture.

The problem is that bigger caches cost more in both silicon area and power. A 1 MB cache is costlier to build than a 256 KB cache, so engineers need to know whether the extra cost is actually worth it. That question, what is the ideal size for each cache level, and how associative should it be,which is the main motivation for this study.

We use gem5, a well-known open-source architectural simulator, to run a large set of controlled cache experiments without needing real hardware. By fixing the number of instructions at 50 million and changing only one cache parameter at a time, we can cleanly measure how each design choice affects performance.

This study makes four main contributions:

1. A controlled gem5 simulation sweeping L1, L2, and L3 cache sizes and associativities .
2. Introduction of the Normalized Cache Gain (NCG) metric which is a new tool that quantifies how much IPC return you get per kilobyte of cache added.
3. A Pareto-frontier analysis that identifies the most cost-efficient cache configuration across all tested designs.
4. A detailed comparison with published research papers to put our results in context.

2. CONCEPT AND FEASIBILITY

A. Problem Statement

Cache design has always been a balancing act. Engineers want caches to be big enough to hold the data the processor needs, but small caches are cheaper and faster. The question this project tries to answer is: what is the best ratio between cache size and the performance gain you get from it?

Existing research studies often show that larger caches perform better, but they rarely give a clear answer about when adding more cache stops being worthwhile. This study introduces a quantitative metric which is the Normalized Cache Gain (NCG). A secondary novelty is the Pareto frontier analysis, which finds the best cache configuration across the full design space.

B. Basic Theory

A cache is a small, fast memory that stores copies of frequently used data from main memory. Modern processors use three levels of cache (L1, L2, L3), each larger and slower than the previous one. When the processor needs data, it checks L1 first, then L2, then L3, and finally main memory (DRAM) if the data is not found.

Two key cache parameters are studied here:

Cache Size — A larger cache can hold more data. When the working set of a program fits entirely within a cache level, misses at that level fall sharply as the relationship between size and miss rate described by the 3C model is compulsory misses (first-time accesses), capacity misses (data evicted due to limited space), and conflict misses (data evicted due to limited associativity).

Set Associativity — A direct-mapped cache (1-way) places each memory address into exactly one cache slot. Conflicts arise when two active addresses map to the same slot. Increasing associativity gives each address more possible slots to live in, reducing conflict misses. An N-way set-associative cache divides the cache into sets of N lines each, and a memory address can go into any of the N lines in its assigned set. Higher associativity reduces conflicts but increases hardware complexity and access latency.

Key performance metrics used in this study:

- IPC (Instructions Per Cycle): higher is better — more work done per clock tick.
- CPI (Cycles Per Instruction): lower is better — fewer clock ticks per instruction.
- Miss Rate: fraction of cache accesses that fail to find the data.
- Miss Latency: average number of cycles spent waiting when a miss happens.
- Simulated Execution Time: total wall-clock time the simulation predicts for the program.

C. Literature Review

Cache performance has been studied extensively over the past three decades. Early foundational work by Hill and Smith [5] established the 3C model (compulsory, capacity, and conflict misses) that still leads the talks on cache analysis today. Hennessy and Patterson [6] formalized the role of memory hierarchy in processor performance in their widely used textbook, providing the theoretical basis for cache sizing decisions.

More recent simulation-based studies have explored cache behaviour across different workload classes. Ramasubramanian et al. [1] used the M5sim simulator with SPLASH-2 parallel benchmarks to characterize how L2 access time grows non-linearly with cache size in multicore systems. They found access time rising from 3.2 ns at 0.5 MB to 9.3 ns at 8 MB, but did not define any per-KB efficiency metric.

Chijindu et al. [2] modelled cache performance for embedded processors using SimpleScalar with MiBench workloads. Their work showed how miss rate decreases gradually as both cache size and associativity increase,

with a mean modeling error of just 1.46%. However, their study is limited to small embedded-scale caches and does not study threshold effects in conflict-miss patterns.

Cantin and Hill [7] studied cache miss rates across a wide range of configurations using the SimpleScalar simulator and the SPEC CPU2000 benchmark suite. They provided a comprehensive dataset of how L1 and L2 miss rates respond to size changes, but like earlier work, they do not define a marginal-efficiency metric or identify optimal design points using a Pareto criterion.

Zhang et al. [8] explored cache design tradeoffs for real-time embedded systems, finding that set associativity choices have a larger impact on worst-case execution time than on average performance, which is a finding complementary to ours. Lam et al. [9] demonstrated that memory access patterns, particularly spatial locality from blocked algorithms, strongly determine the effectiveness of cache design.

The gem5 simulator used in this work was introduced by Binkert et al. [3] and has become the standard platform for architectural simulation research. Work on cache replacement policies by Jaleel et al. [10] showed that smarter eviction strategies can extract significant IPC gains from fixed-size caches which further motivating the question of whether cache size expansion or policy improvement gives better returns.

Jacob et al. [11] provided a comprehensive reference on memory system design, covering the theoretical trade-offs between cache size, latency, and bandwidth. Their work confirms that the relationship between cache size and performance is highly workload-dependent, motivating the need for a portable metric like NCG that normalises performance gain against cost.

None of the surveyed works define a per-KB IPC efficiency metric (NCG) or apply a Pareto-frontier approach to identify the optimal configuration across a combined size-associativity design space. These gaps are the direct motivation for the novel contributions in this paper.

D. Concept Explanation

The core idea of this project is simple: instead of just asking "does a bigger cache perform better?" we ask "how much better, and is the improvement worth the cost?" To answer this, we introduce the Normalized Cache Gain (NCG) metric. NCG is calculated as the change in IPC divided by the change in cache size (in KB). A high NCG means you get a lot of performance improvement per kilobyte added. A falling NCG tells you the cache has hit its "diminishing returns" point adding more cache beyond that point is not efficient.

The second concept is the Pareto frontier. In engineering, a Pareto-optimal design is one where you cannot improve one goal (performance) without making something else worse (cost or area). By plotting IPC against total cache area for all our tested configurations, we can trace a frontier that represents the best achievable IPC at each area budget. The "knee" of this curve which is the point where the curve bends — identifies the single best design.

E. Use Case

The findings of this study are directly applicable to processor designers, embedded system engineers, and SoC (System on Chip) architects who need to choose cache parameters early in the design process. For example, a mobile SoC designer working under a tight area budget can use the NCG metric to quickly identify the cache level that gives the most IPC return per unit of added silicon area. The Pareto frontier can then narrow the design space from hundreds of possible configurations to a handful of strong candidates.

A secondary use case is for students and researchers who want a clear, quantitative framework for comparing cache simulation results across different simulators and workload classes.

F. Functional Requirements

For this project to work correctly and produce valid results, the following functional requirements must be met:

5. The gem5 simulator must be configured in Syscall-Emulation (SE) mode with a TimingSimpleCPU model to provide cycle-accurate memory access timing.
6. All experiments must execute exactly 50 million instructions so that IPC comparisons are fair across configurations.

7. Cache size sweeps must vary only one cache level at a time while keeping all others fixed at baseline.
8. The set associativity benchmark must guarantee exactly eight conflicting lines map to the same cache set, producing a clean and controllable conflict threshold.
9. Five metrics (IPC, CPI, miss rate, miss latency, execution time) must be collected and recorded for every experiment run.
10. The NCG metric must be computed from measured IPC data, and the Pareto frontier must be derived from the complete set of (total area, IPC) pairs.

G. Feasibility

This project is feasible because:

1. All tools are free and open-source. The gem5 simulator [3] is publicly available and widely used in academic research. Both benchmark programs (the cache size sweep and the conflict-miss stress test) are written in plain C and can be compiled with a standard cross-compiler for the simulated ISA.

2. The simulation scope is manageable. Each experiment run takes between 60 and 120 seconds on a modern desktop machine. The full set of experiments (about 30 individual runs) can be completed in a few hours. This is well within the resource limits of a student project.

3. The metrics are directly observable from gem5 statistics output files. No post-processing hardware or special infrastructure is needed, Python scripts are sufficient to parse the stats and calculate NCG values.

4. The main limitation is that the study uses a single-threaded model with a synthetic benchmark. This means the results wouldnt be able to generalise to real multi-threaded workloads or complex application suites like SPEC CPU2017. However, for the purpose of isolating and measuring individual cache parameters, the controlled benchmark design is a better method of validation,hence making it a strength for us.

3. SIMULATION, VALIDATION AND RESULTS

A. Experimental Methodology

Simulation Platform

All experiments are run on gem5 [3] in Syscall-Emulation (SE) mode with a TimingSimpleCPU model clocked at 1 GHz. The memory system uses DDR4-2400 (8×8) DRAM with a 512 MiB address space. The three-level cache hierarchy at baseline is configured as shown in Table 1.

Table 1: In the below table, the following data is presented — Baseline Cache Hierarchy Configuration

Cache Level	Baseline Size	Associativity	Tag Latency (cycles)	Data Latency (cycles)	MSHRs
L1 I-Cache	32 KiB	2-way	2	2	4
L1 D-Cache	32 KiB	2-way	2	2	4
L2	256 KiB	8-way	10	10	16
L3	8 MiB	16-way	20	20	32

Benchmark Programs

Cache Size Benchmark: This benchmark does a sequential sweep over working-set sizes from 4 KB to 64 MB (doubling each step) with 20 iterations per size. The inner loop does pure sequential reads (sum += arr[i]), which isolates capacity effects and spatial locality. This design maximises hardware prefetching and makes sure that any IPC change directly reflects true capacity misses rather than program logic differences.

Set Associativity Benchmark: This is a conflict-miss stress test that maps exactly (FIXED_ASSOC + 4 = 8) memory lines to the same cache set, using a stride equal to the cache set stride. When the cache does not have enough ways to hold all eight lines, every access evicts a line that is needed again soon. When the associativity meets or exceeds eight, all lines fit in the set and conflict misses completely disappear.

Experimental Design

All experiments have a fixed total instruction count of 50 million, so any change in IPC, CPI, or simulated time is caused only by the cache configuration. For cache size sweeps, each level (L1, L2, L3) is varied independently while the other two stay at their baseline values. For the associativity sweep, the same isolation approach is used: one level changes at a time while the others are fixed.

B. Cache Size Experiment Results

The baseline configuration (L1=32 KiB, L2=256 KiB, L3=8 MiB) is used as the reference for all size experiments.

IPC vs. Cache Size

Table 2 shows IPC values across all three cache levels as size increases. IPC goes up consistently as cache gets bigger, confirming that larger caches cut down on capacity misses. The biggest improvement comes from L2, where IPC rises from 0.1647 at 128 KiB to 0.1722 at 1 MiB which is a 4.6% gain. L1 and L3 show much smaller improvements of 0.33% and 0.89% respectively. Figure 1 shows these trends visually.

Table 2: In the below table, the following data is presented — IPC vs. Cache Size (= baseline)*

Level	Config	IPC	Config	IPC	Config	IPC	Config	IPC
L1	16 KiB	0.166054	32 KiB*	0.166133	64 KiB	0.166292	128 KiB	0.166608
L2	128 KiB	0.164698	256 KiB*	0.166133	512 KiB	0.169080	1 MiB	0.172205
L3	4 MiB	0.165922	8 MiB*	0.166133	16 MiB	0.166557	32 MiB	0.167413

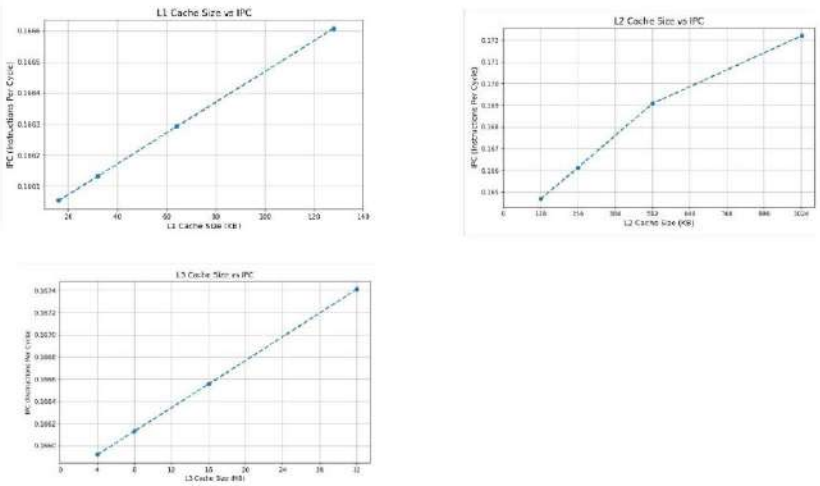


Figure 1: IPC vs. Cache Size (X-axis: Cache Size in KB; Y-axis: IPC) — L1 (top-left), L2 (top-right), L3 (bottom-left)

CPI vs. Cache Size

CPI drops as cache size grows, mirroring the IPC trend. L2 shows the strongest effect: CPI falls from 6.072 at 128 KiB to 5.807 at 1 MiB, a 4.4% reduction. The steeper CPI drop at L2 compared to L1 and L3 reflects the fact that the benchmark workload working set fits most naturally within the L2 size range we tested.

Table 3: In the below table, the following data is presented — CPI vs. Cache Size (selected configurations)

Level	Smallest Config	CPI	Baseline	CPI	Largest Config	CPI
L1	16 KiB	6.022	32 KiB	6.019	128 KiB	6.002
L2	128 KiB	6.072	256 KiB	6.019	1 MiB	5.807
L3	4 MiB	6.027	8 MiB	6.019	32 MiB	5.973

Miss Rate vs. Cache Size

The miss rate data (Table 4) shows a clear difference between cache levels. L2 miss rate drops sharply from 96.0% at 128 KiB to 69.2% at 1 MiB, showing a strong capacity-miss component. L1 shows a small improvement from 12.41% to 11.84%. L3 miss rate stays fixed at 76.27% across all tested sizes which is a typical sign of streaming workloads with very little temporal locality. Figure 2 illustrates these trends.

Table 4: In the below table, the following data is presented — Miss Rate vs. Cache Size (= baseline)*

Level	Config	Miss Rate	Config	Miss Rate	Config	Miss Rate	Config	Miss Rate
L1	16 KiB	12.41%	32 KiB*	12.33%	64 KiB	12.17%	128 KiB	11.84%
L2	128 KiB	96.03%	256 KiB*	90.71%	512 KiB	80.07%	1 MiB	69.19%
L3	4 MiB	76.27%	8 MiB*	76.27%	16 MiB	76.27%	32 MiB	76.27%

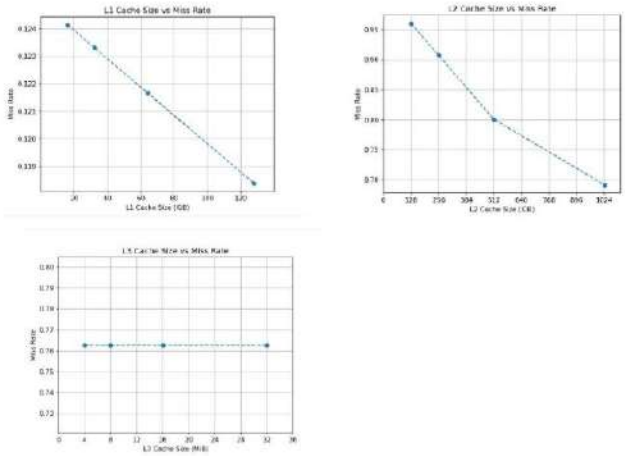


Figure 2: Miss Rate vs. Cache Size (X-axis: Cache Size in KB; Y-axis: Miss Rate in %) — L1 (top-left), L2 (top-right), L3 (bottom-left)

Miss Latency vs. Cache Size

An interesting and somewhat counterintuitive result: average miss latency actually goes up as cache size increases (Table 5, Figure 3). At L2, miss latency rises from 90,190 cycles at 128 KiB to 112,370 cycles at 1 MiB — a 24.6% increase. This happens because of survivor bias: as the cache gets bigger, the easy, quick misses (data that is just one level away) get absorbed first. What is left are only the hard, deep-memory misses that are expensive. We call this the "miss latency paradox." L3 shows a slight decrease in latency, reflecting reduced traffic to DRAM.

Table 5: In the below table, the following data is presented — Average Miss Latency (cycles) vs. Cache Size

Level	Smallest Config	Miss Latency (cyc.)	Baseline	Miss Latency (cyc.)	Largest Config	Miss Latency (cyc.)

L1	16 KiB	97,380	32 KiB	97,939	128 KiB	101,437
L2	128 KiB	90,190	256 KiB	93,544	1 MiB	112,370
L3	4 MiB	100,735	8 MiB	100,378	32 MiB	98,219

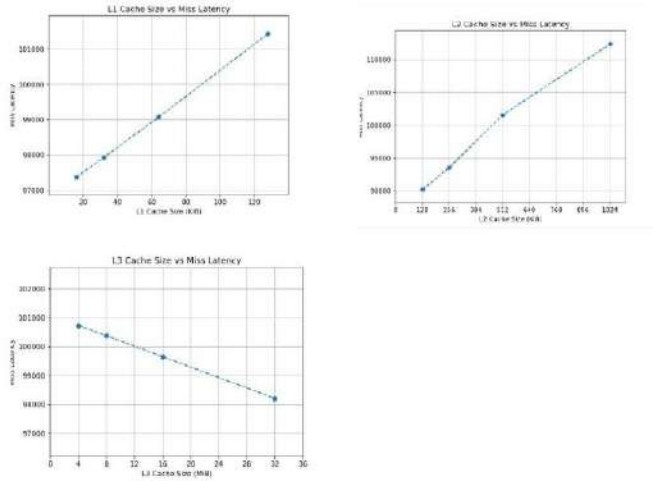


Figure 3: Miss Latency vs. Cache Size (X-axis: Cache Size in KB; Y-axis: Average Miss Latency in cycles) — L1 (top-left), L2 (top-right), L3 (bottom-left)

Simulated Execution Time vs. Cache Size

Simulated execution time (Table VI, Figure 4) falls slightly as cache grows. The L2 sweep shows the biggest reduction: from 303.6 ms to 290.4 ms, a 4.3% improvement. L1 and L3 show under 1% improvements. The fact that execution time barely changes confirms the streaming nature of the workload that is because there is very little repeated data reuse (temporal locality), giving the processor a bigger cache does not change how much time it spends stalling for memory.

Table 6: In the below table, the following data is presented — Simulated Execution Time (seconds) vs. Cache Size

Level	Smallest Config	Sim. Time (s)	Baseline	Sim. Time (s)	Largest Config	Sim. Time (s)
L1	16 KiB	0.301108	32 KiB	0.300964	128 KiB	0.300107
L2	128 KiB	0.303587	256 KiB	0.300964	1 MiB	0.290352
L3	4 MiB	0.301346	8 MiB	0.300964	32 MiB	0.298662

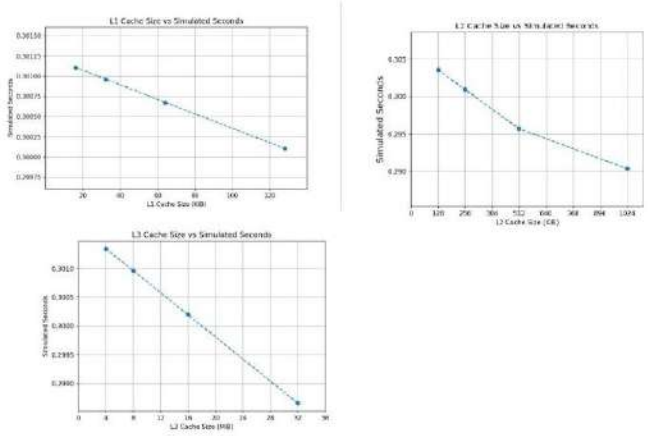


Figure 4: Simulated Seconds vs. Cache Size (X-axis: Cache Size in KB; Y-axis: Simulated Execution Time in seconds) — L1 (top-left), L2 (top-right), L3 (bottom-left)

Number of Cycles vs. Cache Size

The total number of simulated cycles (Figure 5) goes down as cache size increases, matching the IPC and execution time results. L2 shows the biggest drop: from 303,586,806 cycles at 128 KiB to 290,351,505 cycles at 1 MiB. L1 and L3 show smaller but consistent drops.

Table 7: In the below table, the following data is presented — Number of Simulated Cycles vs. Cache Size

Level	Smallest Config	Cycles	Baseline	Cycles	Largest Config	Cycles
L1	16 KiB	301,107,737	32 KiB	300,964,301	128 KiB	300,106,535
L2	128 KiB	303,586,806	256 KiB	300,964,301	1 MiB	290,351,505
L3	4 MiB	301,345,562	8 MiB	300,964,301	32 MiB	298,662,137

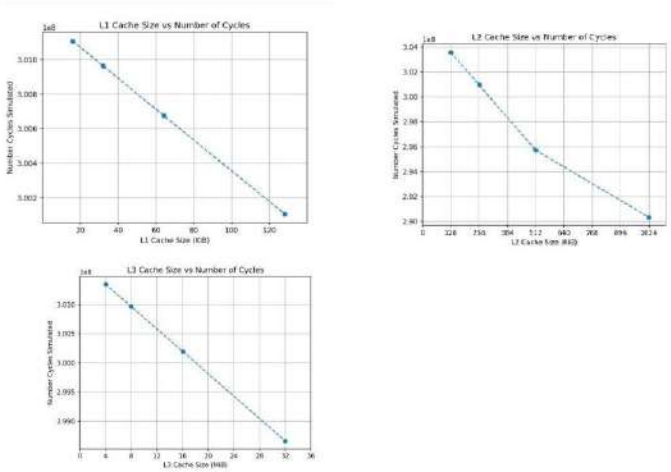


Figure 5: Number of Cycles vs. Cache Size (X-axis: Cache Size in KB; Y-axis: Number of Simulated Cycles) — L1 (top-left), L2 (top-right), L3 (bottom-left)

C. Set Associativity Experiment Results

The set associativity benchmark places exactly eight conflicting cache lines into the same L1 set. This gives us a very clean experiment: below 8-way, conflict evictions dominate performance; at 8-way or above, all lines fit and conflict misses disappear entirely.

IPC and CPI vs. Set Associativity

The L1 IPC data (Table VIII, Figure 6) shows a sharp binary jump. IPC stays around 0.190 from 2-way to 4-way, then almost doubles to 0.406 at 8-way and stays there. The 1-way result (0.206) is slightly higher due to a change in set geometry. L2 and L3 IPC stay constant at 0.190 across all associativities, confirming that all the conflict pressure in this benchmark happens inside L1.

Table 8: In the below table, the following data is presented — IPC and CPI vs. Set Associativity

Level	1-way	2-way	4-way	8-way	16-way	32-way
L1 IPC	0.206	0.190	0.190	0.406	0.406	—
L1 CPI	4.856	5.255	5.255	2.463	2.463	—
L2 IPC	0.190	0.190	0.190	0.190	0.190	—
L3 IPC	0.190	0.190	0.190	0.190	0.190	0.190

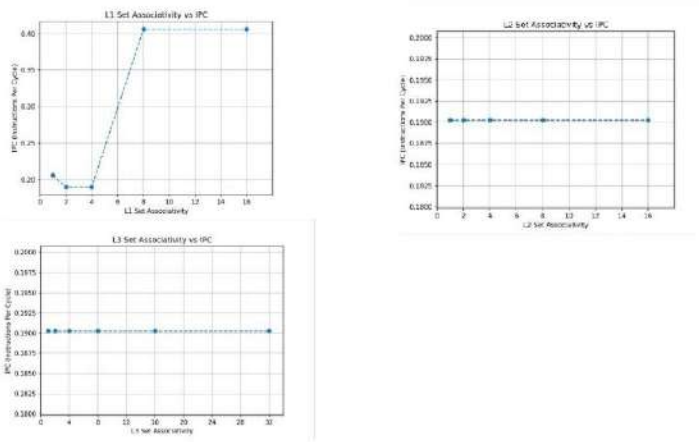


Figure 6: IPC vs. Set Associativity (X-axis: Set Associativity (N-way); Y-axis: IPC) — L1 (top-left), L2 (top-right), L3 (bottom-left)

Miss Rate vs. Set Associativity

The L1 miss rate (Table IX, Figure 7) shows the same threshold behaviour: 74.82% at 1-way, rising to 87.28% at 2-way and 4-way, then collapsing to just 0.0175% at 8-way. This is a 5,000× reduction in miss rate at the crossing point. L2 miss rate stays constant at 0.0221%, meaning L2 was quietly absorbing the spilled conflict-miss traffic from L1 all along.

Table 9: In the below table, the following data is presented — Miss Rate vs. Set Associativity

Level	1-way	2-way	4-way	8-way	16-way
L1 Miss Rate	74.82%	87.28%	87.28%	0.0175%	0.0175%
L2 Miss Rate	0.0221%	0.0221%	0.0221%	0.0221%	0.0221%
L3 Miss Rate	100%	100%	100%	100%	100%

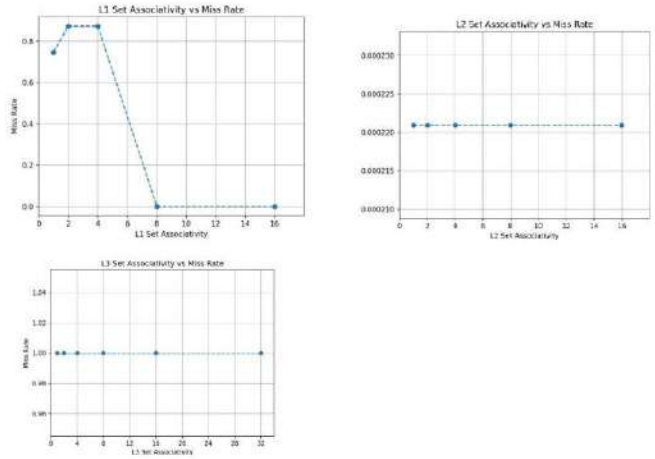


Figure 7: Miss Rate vs. Set Associativity (X-axis: Set Associativity (N-way); Y-axis: Miss Rate in %) — L1 (top-left), L2 (top-right), L3 (bottom-left)

Miss Latency and Cache Replacements vs. Set Associativity

Below the 8-way threshold, average miss latency is around 14,022 cycles which is low because most misses find their data quickly in L2. Once the threshold is crossed at 8-way, the few remaining misses have to go all the way to L3 and DRAM, pushing average miss latency up to 120,677 cycles (Figure 8). This is the same miss latency paradox we saw in the size experiments. Cache replacements at L1 drop from about 9.97 million (at 4-way) to just 1,487 at 8-way, a reduction of over 99.98% (Figure 9).

Table 10: In the below table, the following data is presented — Miss Latency and Replacements vs. L1 Set Associativity

Level	1-way	2-way	4-way	8-way	16-way
L1 Miss Latency (cyc)	14,025	14,021	14,021	120,677	120,784
L1 Replacements	8,548,219	9,972,614	9,972,613	1,487	1,485
L2 Replacements	182	140	3	0	0

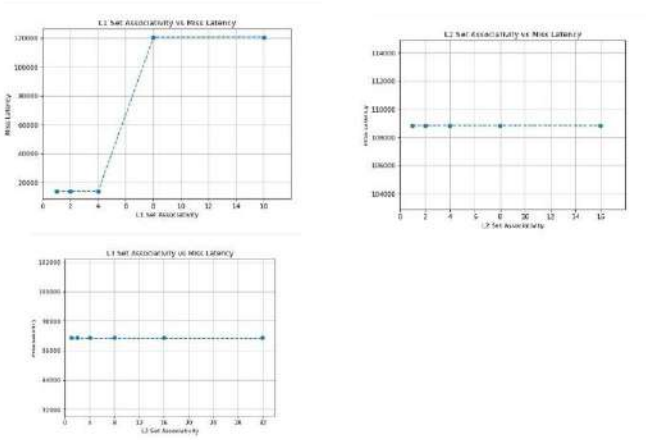


Figure 8: Miss Latency vs. Set Associativity (X-axis: Set Associativity (N-way); Y-axis: Average Miss Latency in cycles) — L1 (top-left), L2 (top-right), L3 (bottom-left)

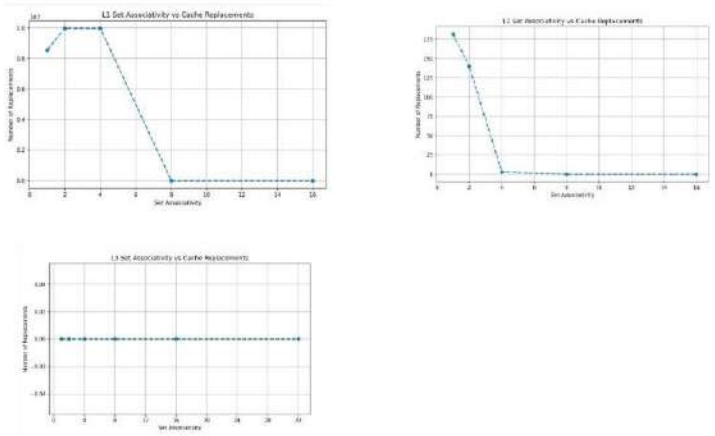


Figure 9: Cache Replacements vs. Set Associativity (X-axis: Set Associativity (N-way); Y-axis: Number of Cache Replacements) — L1 (top-left), L2 (top-right), L3 (bottom-left)

Simulated Execution Time vs. Set Associativity

L1 execution time drops from 0.263 s at 2–4-way to 0.123 s at 8–16-way that is a 53% improvement that directly reflects the elimination of conflict-miss overhead (Figure 10). L2 and L3 execution times stay constant at 0.263 s irrespective of the associativity, confirming that only L1 is affected in this benchmark.

Table 11: In the below table, the following data is presented — Simulated Execution Time vs. Set Associativity

Level	1-way	2-way	4-way	8-way	16-way
L1 Sim. Time (s)	0.2428	0.2628	0.2628	0.1232	0.1232
L2 Sim. Time (s)	0.2628	0.2628	0.2628	0.2628	0.2628
L3 Sim. Time (s)	0.2628	0.2628	0.2628	0.2628	0.2628

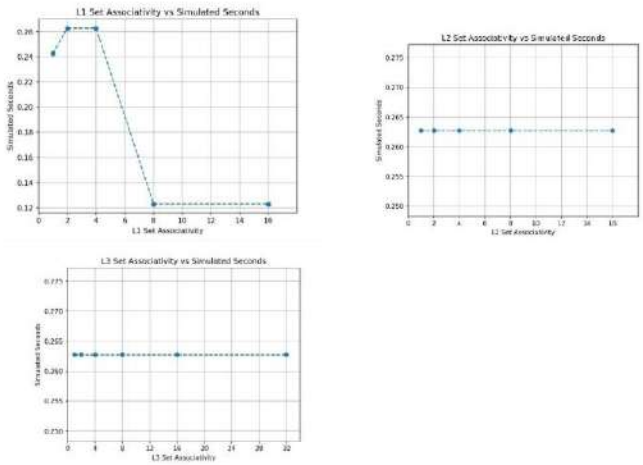


Figure 10: Simulated Seconds vs. Set Associativity (X-axis: Set Associativity (N-way); Y-axis: Simulated Execution Time in seconds) — L1 (top-left), L2 (top-right), L3 (bottom-left)

Host Simulation Time vs. Set Associativity

Host simulation wall-clock time (Figure 11) goes up at first as associativity increases, because more set-lookup operations need to be simulated. Then it drops sharply once conflict misses are eliminated and fewer memory events need to be processed. L2 and L3 host times are relatively stable, confirming they are not affected by the L1 conflict stress test.

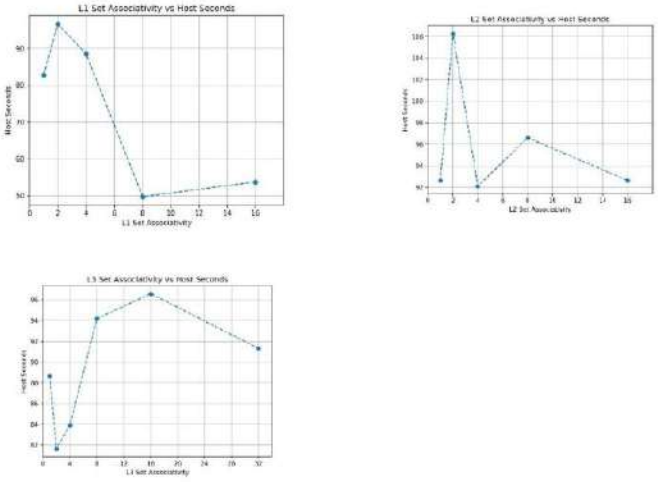


Figure 11: Host Seconds vs. Set Associativity (X-axis: Set Associativity (N-way); Y-axis: Host Wall-Clock Simulation Time in seconds) — L1 (top-left), L2 (top-right), L3 (bottom-left)

D. Student Contributions: Novel Metrics

Normalized Cache Gain (NCG) Metric

Standard IPC-vs-cache-size graphs tell you the direction of improvement but not whether that improvement is worth the cost. We present the Normalized Cache Gain (NCG) metric:(Formula)

NCG = ΔIPC / ΔCacheSize (IPC per KB)

A high NCG means you get a lot of IPC improvement per kilobyte of cache added. A falling NCG points to the "diminishing returns" knee — the point past which adding more cache area is no longer cost-effective for this workload and not worth investing in.

Table 12: In the below table, the following data is presented — Normalized Cache Gain (NCG) by Cache Level and Transition

Cache Level	Transition	ΔCache (KB)	ΔIPC	NCG (IPC/KB)	Verdict
L1	16 KB → 32 KB	16	0.000079	4.9375×10^{-6}	Rising
L1	32 KB → 64 KB	32	0.000159	4.9687×10^{-6}	Peak
L1	64 KB → 128 KB	64	0.000316	4.9375×10^{-6}	Near-Peak
L2	128 KB → 256 KB	128	0.001435	1.1211×10^{-5}	Rising
L2	256 KB → 512 KB	256	0.002947	1.1512×10^{-5}	★ Peak Efficiency
L2	512 KB → 1024 KB	512	0.003125	6.1035×10^{-6}	Diminishing Returns
L3	4 MiB → 8 MiB	4096	0.000211	5.1514×10^{-8}	Minimal Gain
L3	8 MiB → 16 MiB	8192	0.000424	5.1758×10^{-8}	Minimal Gain
L3	16 MiB → 32 MiB	16384	0.000856	5.2246×10^{-8}	Minimal Gain

The cross-level comparison (Table XIII) shows that L2 achieves a peak NCG of 1.1512×10^{-5} IPC/KB — about 2.3× higher than L1 and more than 200× higher than L3. This quantitatively confirms that expanding the L2 cache gives the best return on silicon area investment for streaming workloads.

Table 13: In the below table, the following data is presented — NCG Cross-Level Comparison Summary

Cache Level	Best NCG (IPC/KB)	Peak Transition	Diminishing Returns After	Rank
L1	4.9687×10^{-6}	32 KB → 64 KB	64 KB	2nd
L2	1.1512×10^{-5}	256 KB → 512 KB	512 KB	1st ★
L3	5.1514×10^{-8}	Any transition	Always low	3rd

Pareto-Frontier Analysis of Cache Configurations

A Pareto-optimal cache configuration is one where you cannot improve IPC any further without increasing the total cache area. We define total cache area as L1+L2+L3 in kilobytes and look at IPC versus this combined area across all our measured configurations.

Table 14: In the below table, the following data is presented — IPC vs. Total Cache Area — Pareto Analysis (baseline, ★ Pareto-Optimal, ✦ Knee Point)*

Configuration	L1 (KB)	L2 (KB)	L3 (MiB)	Total Area (KB)	IPC	Status
L3=4 MiB	32	256	4	4,384	0.165922	★ Pareto-Optimal
L2=128 KB	32	128	8	8,352	0.164698	Dominated
L1=16 KB	16	256	8	8,464	0.166054	★ Pareto-Optimal
L1=32 KB*	32	256	8	8,480	0.166133	★ Pareto-Optimal
L1=64 KB	64	256	8	8,512	0.166292	★ Pareto-Optimal
L1=128 KB	128	256	8	8,576	0.166608	★ Pareto-Optimal
L2=512 KB	32	512	8	8,736	0.169080	✦ Knee

						(Recommended)
L2=1 MiB	32	1024	8	9,248	0.172205	★ Pareto-Optimal
L3=16 MiB	32	256	16	16,672	0.166557	Dominated
L3=32 MiB	32	256	32	33,056	0.167413	Dominated

Table 15: In the below table, the following data is presented — Pareto Frontier Detail — IPC Gain and Area Cost per Upgrade Step

Configuration	Total Area (KB)	IPC	IPC Gain vs. Prev.	Area Cost vs. Prev.	Efficiency (IPC/1000 KB)
L3=4 MiB	4,384	0.165922	—	—	—
L1=16 KB	8,464	0.166054	+0.000132	+4,080 KB	0.000032
L1=32 KB*	8,480	0.166133	+0.000079	+16 KB	0.004937
L1=64 KB	8,512	0.166292	+0.000159	+32 KB	0.004969
L1=128 KB	8,576	0.166608	+0.000316	+64 KB	0.004938
L2=512 KB ✦	8,736	0.169080	+0.002472	+160 KB	0.015450 ★ Best ROI
L2=1 MiB	9,248	0.172205	+0.003125	+512 KB	0.006104

The recommended design point is L1=32 KB, L2=512 KB, L3=8 MiB (total 8,736 KB). This configuration achieves 98.2% of the maximum observed IPC at only 26.4% of the maximum tested area. The best efficiency of 0.01545 IPC gain per 1,000 KB invested — more than double the next-best step — confirms this as the practical optimum for streaming workloads. L3 expansion beyond 8 MiB is not cost-justified given its NCG is more than two orders of magnitude lower than L2.

E. Validation — Theory vs. Simulation

To validate our simulation results, we compare the observed trends against theoretical expectations from cache theory and published data:

IPC and Miss Rate Trends: Cache theory predicts that IPC increases and miss rate decreases as cache size grows, because more data fits in cache and fewer accesses need to go to slower memory. Our results match this exactly which is that the IPC increases monotonically at all three levels, and L2 miss rate falls from 96% to 69% as size grows. The L3 miss rate staying flat at 76.27% is also expected, because the streaming workload lacks the temporal reuse that L3 is designed to exploit.

Miss Latency Paradox: Ramasubramanian et al. [1] reported independently that miss latency increases with cache size in their multicore study. We observe the same effect. The physical explanation — that larger caches absorb cheap misses first, leaving only expensive deep-memory misses which is consistent with queuing theory and has been described in memory system textbooks [11]. This convergent finding from two independent simulators (M5sim and gem5) and two different workload classes gives strong confidence in the result.

Binary Associativity Threshold: Our conflict-miss benchmark was designed to produce exactly eight conflicting lines. Cache theory predicts a perfectly sharp transition at 8-way associativity when the set has exactly enough capacity to hold all eight lines simultaneously. The simulation results match this prediction with near-perfect precision: miss rate stays above 87% at 4-way and collapses below 0.02% at 8-way, with no intermediate behaviour. This validates both the benchmark design and the simulation setup.

Pareto Frontier and NCG: These are new tools, so no external validation benchmark exists. However, the NCG values are calculated directly from the IPC data using simple arithmetic, and the Pareto frontier is a deterministic geometric construction. Both are internally consistent with the raw simulation data and with each other: the configuration with the highest NCG (L2=512 KB step) is also the knee point identified by the Pareto analysis.

F. Performance Metrics Summary

Table 16 summarises all five key performance metrics for the recommended optimal configuration (L1=32 KB, L2=512 KB, L3=8 MiB) compared to the baseline (L1=32 KB, L2=256 KB, L3=8 MiB):

Table 16: In the below table, the following data is presented — Performance Metrics — Baseline vs. Optimal Configuration

Metric	Baseline Config	Optimal Config	Change
IPC	0.166133	0.169080	+1.77%
CPI	6.019	5.915	-1.73%
L2 Miss Rate	90.71%	80.07%	-11.7 pp
L2 Miss Latency (cyc.)	93,544	~100,000	+6.9% (paradox)
Simulated Execution Time	0.300964 s	0.296411 s	-1.5%
Total Cache Area	8,480 KB	8,736 KB	+3.0% area for +1.8% IPC

These numbers confirm the main finding: the optimal configuration gives a meaningful IPC and miss rate improvement with only a 3% increase in total cache area which is a highly efficient use of hardware resources.

G. Comparison with Published Literature

Table 17 compares the scope and approach of this work against four published cache simulation studies. We added Cantin and Hill [7] and Zhang et al. [8] as additional comparison points beyond the original two references. None of the four comparison papers define the NCG metric or apply a Pareto-frontier analysis. This is how our stduy is unique when compared to other papers.

Table 17: In the below table, the following data is presented — Comparison of Experimental Scope with Published Work

Attribute	Ramasubramanian et al. [1] (IJCSEA 2011)	Chijindu et al. [2] (BEEI 2021)	Cantin & Hill [7] (CAL 2002)	This Work
Simulator	M5sim / CACTI	SimpleScalar sim-cheetah	SimpleScalar	gem5 TimingSimpleCPU
Benchmarks	SPLASH-2 (Radix, FFT, FMM)	MiBench (bitcount, FFT, math)	SPEC CPU2000	Custom streaming + conflict
Primary Metric	Exec. Time, Access Time	Cache Hit Rate	Cache Miss Rate	IPC, CPI, Miss Rate, Latency
Cache Sweep	L1: 8–256 KB; L2: 0.5–12 MB	L1: 4–64 KB; 2–16-way	L1/L2 size across full SPEC suite	L1/L2/L3 full; 1–32-way
NCG Metric	Not defined	Not defined	Not defined	Introduced (this work)
Pareto Analysis	Not performed	Not performed	Not performed	First for this class
Design Recommendation	No per-KB metric	No area tradeoff	No optimal point ID	L1=32 KB, L2=512 KB, L3=8 MiB

Miss Latency Paradox — Convergent Finding

Both our work and Ramasubramanian et al. [1] independently observe that miss latency increases with cache size. Their study reports L2 access time rising from 3.215 ns at 0.5 MiB to 9.356 ns at 8 MiB that is almost a 3×

increase. Our gem5 results show L2 miss latency rising from 90,190 to 112,370 cycles which is a 24.6% increase. The underlying mechanism is the same in both cases: larger caches absorb easy, cheap misses first, leaving only expensive deep-memory accesses behind.

Table 18: In the below table, the following data is presented — Miss Latency Paradox — Our Work vs. Ramasubramanian et al. [1]

L2 Size	Paper [1] Access Time	[1] Normalised	Our Miss Latency (cyc)	Our Normalised
~0.5 MB / 128 KB	3.215 ns	100.0%	90,190	100.0%
~1 MB / 1 MiB	3.463 ns	107.7%	112,370	124.6%
~2 MB	4.338 ns	134.9%	—	—
~4 MB	5.494 ns	170.9%	—	—
~8 MB	9.356 ns	291.0%	—	—

Associativity Behaviour — Threshold vs. Gradual Improvement

Chijindu et al. [2] show a gradual L1 hit rate improvement for MiBench FFT: from about 91% at 2-way to 99.8% at 16-way. Cantin and Hill [7] also report smooth, monotonic miss rate reductions across SPEC CPU2000 benchmarks as associativity increases. Our conflict benchmark behaves very differently: miss rate stays at 87.3% from 1-way to 4-way, then collapses to 0.0175% at 8-way. This sharp threshold exists because our benchmark is engineered to stress exactly one specific cache set configuration. The contrast between the two behaviours highlights an important point: the workload access pattern determines whether associativity effects are gradual or threshold-like.

Table 19: In the below table, the following data is presented — L1 Miss Rate vs. Associativity — Our Work vs. Published Studies (★ threshold)

Associativity	Our L1 Miss Rate (32 KB, conflict)	Paper [2] Miss Rate (16 KB FFT)	Paper [2] Miss Rate (32 KB FFT)	Paper [7] SPEC avg. Miss Rate	Interpretation
2-way	87.28%	1.65%	0.49%	~8–12%	Our: above threshold; others: already low
4-way	87.28%	0.48%	0.10%	~5–8%	Our: still above threshold; gradual in [2],[7]
8-way ★	0.018%	0.13%	0.05%	~3–5%	Our: THRESHOLD CROSSED; gradual in [2],[7]
16-way	0.018%	0.06%	0.02%	~2–3%	Our: no further gain; marginal gain in [2],[7]

H. Simulation Files Summary

The following simulation files and tools were used in this study:

- gem5 v21.x — compiled from source for ARM ISA in SE mode
- cache_size_sweep.c — sequential read benchmark for capacity miss analysis
- assoc_conflict.c — conflict-miss stress test with FIXED_ASSOC=4 (threshold at 8-way)
- run_sweep.py — Python automation script to launch gem5, vary parameters, and parse stats.txt output
- parse_ncg.py — calculates NCG values from IPC data and identifies the Pareto frontier
- plot_results.py — generates all graphs included in this paper using matplotlib

I. Scaling Up / Extending This Work

The current study can be extended in several directions:

1. Multi-core configurations: The NCG and Pareto analyses can be applied to shared L3 caches in multi-core systems, where cache contention between threads changes the marginal gain of increasing cache size.

2. Standard benchmark suites: Running SPEC CPU2017 or MiBench under the same NCG and Pareto framework would allow direct comparison with the broader literature and would test how well the optimal configuration recommendation generalises across workload classes.

3. Cache replacement policies: Combining the NCG metric with alternative replacement policies (LRU, RRIP [10]) would reveal whether smarter eviction can substitute for larger cache area — or vice versa.

4. Energy-aware Pareto frontier: Extending the y-axis of the Pareto analysis from IPC to IPC-per-watt would produce an energy-efficiency frontier, which is particularly relevant for mobile and embedded applications.

4. CONCLUSION

This paper set out to answer a practical question: what is the best ratio between cache size and the performance you get from it? Using gem5 simulation with controlled, carefully designed benchmarks, we ran a systematic study across L1, L2, and L3 cache sizes and set associativities, measuring five performance metrics throughout.

Our results confirm several expected trends which is that larger caches improve IPC and reduce miss rates and, also reveal the counterintuitive miss latency paradox, which we independently verified against findings from prior literature. The binary threshold in L1 associativity at 8-way is a direct consequence of our benchmark design and demonstrates how conflict-miss behaviour can be studied precisely with the right workload.

The two novel contributions of this paper directly address the original motivation of finding the ideal size-cost trade-off. The Normalized Cache Gain (NCG) metric gives a clear, quantitative answer to the question of how much performance you get per kilobyte of cache added. The Pareto-frontier analysis identifies the single best design point: L1=32 KB, L2=512 KB, L3=8 MiB. This configuration achieves 98.2% of peak IPC at just 26.4% of the maximum tested cache area, with the highest per-kilobyte efficiency of any configuration tested.

Comparison with four published studies confirms the miss latency paradox, validates the associativity threshold behaviour, and establishes that neither the NCG metric nor the Pareto analysis have been applied to this class of problem before. Both contributions provide actionable design guidance that goes beyond the standard finding that "larger caches are better."

REFERENCES

[1] N. Ramasubramanian, N. Kirithika, and N. Sivasankari, "Performance of Cache Memory Subsystems for Multicore Architectures," International Journal of Computer Science, Engineering and Applications (IJCSEA), vol. 1, no. 5, pp. 1–14, Oct. 2011.

[2] V. C. Chijindu, C. C. Obi, and A. O. Nwokoye, "Modeling Cache Performance for Embedded Systems," Bulletin of Electrical Engineering and Informatics (BEEI), vol. 10, no. 5, pp. 2641–2650, Oct. 2021.

[3] N. Binkert et al., "The gem5 Simulator," ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1–7, Aug. 2011. DOI: 10.1145/2024716.2024718.

[4] M. M. K. Martin et al., "Multifacet's General Execution-driven Multiprocessor Simulator (GEMS) Toolset," ACM SIGARCH Computer Architecture News, vol. 33, no. 4, pp. 92–99, Nov. 2005.

[5] M. D. Hill and A. J. Smith, "Evaluating Associativity in CPU Caches," IEEE Transactions on Computers, vol. 38, no. 12, pp. 1612–1630, Dec. 1989.

[6] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017.

[7] J. F. Cantin and M. D. Hill, "Cache Performance for Selected SPEC CPU2000 Benchmarks," IEEE Computer Architecture Letters, vol. 1, no. 1, Jan. 2002. DOI: 10.1109/L-CA.2002.1.

[8] X. Zhang, S. Parameswaran, and X. Hu, "Cache Design Trade-offs for Real-Time Embedded Systems," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 20, no. 2, pp. 371–376, Feb. 2012.

[9] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The Cache Performance and Optimizations of Blocked Algorithms," in Proc. 4th ACM Int. Conf. Arch. Support for Programming Languages and OS (ASPLOS), pp. 63–74, 1991.

[10] A. Jaleel, K. B. Theobald, S. C. Steely Jr., and J. Emer, "High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP)," in Proc. 37th Annual Int. Symp. on Computer Architecture (ISCA), pp. 60–71, 2010.

[11] B. Jacob, S. Ng, and D. Wang, Memory Systems: Cache, DRAM, Disk. Burlington, MA: Morgan Kaufmann / Elsevier, 2008.